

PAPER

Measuring the Gap Between Local Call Savings and End-to-End Latency in Multi-Agent RAG

SUMMARY Removing a repeated model call can reduce local work without a detectable change in complete response time. We measure this gap in a multi-agent retrieval-augmented generation (RAG) system using original, unsigned direct, and signed direct dispatch of prepared search arguments. Three rotated rounds of the same 200 legal questions give 1,800 requests at concurrency four. All 648 original search branches preserve their prepared arguments. Direct dispatch removes these rewrite calls, reduces non-handler boundary time from 543.140 ms to 0.521 ms, and lowers requests to the tool-serving engine by 41% per batch, but prompt tokens by only 1–2%. For requests that use search, signed dispatch shortens the time from request start to the last search completion by 0.46–0.64 seconds in every round, whereas original-versus-original controls at the same scope change by at most 0.12 seconds. Full-response means change direction across rounds: the position-balanced estimate is a 0.123-second decrease with a question-bootstrap interval of $[-1.15, 1.29]$ seconds, and original rounds differ from each other by up to 1.5 seconds. Source-set agreement between arms (89–92%) is below each arm's own round-to-round agreement (95%); a follow-up with two instances per arm shows that instances of the same arm agree no more than instances of different arms (83–87%) and that the search handler returns identical law lists for identical inputs in only 65–79% of cross-instance cases, so this gap is retrieval nondeterminism, not dispatch. A rotated load sweep at concurrency 8–100 (4,000 requests) shows why: the removed rewrite call grows from 1.2 to 22 seconds per branch under load and the tool engine's queue time per request falls by 29–72%, yet the orchestrator engine saturates from concurrency 32 and bounds the end-to-end saving to 1.4–2.9% below saturation and 2.3–2.6% above it. Per-question sign tests favor direct dispatch at every level except 32, while mean intervals mostly include zero. Local cost, intermediate completion, full-response time, source agreement, and load dependence must each be measured against a matched control.

key words: multi-agent RAG, latency measurement, tool calling, experimental design, retrieval agreement

1. Introduction

Retrieval-augmented generation (RAG) combines document retrieval with language generation[1]. Multi-agent search divides this work among preparation, search, evidence selection, and answer generation. Some model calls make new decisions; others reformat information already available to the application. Separating these roles helps identify work that ordinary code can perform.

We study a fixed search boundary in a legal search application. A preparation stage supplies a country, search terms, and keywords. The original path sends these values to a tool-calling model with one registered search operation. The adapter passes the prepared arguments directly to the same handler. Other models, prompts, search data, and serving settings remain unchanged.

A fixed operation does not make rewriting unnecessary: the model might correct inputs. We check prepared/executed arguments and final law sets separately; neither is expert

review.

The main question is whether a measured local saving survives at the request level. Branch work can overlap, and later calls add variable delays. A large relative improvement in a short step can therefore be misleading when presented as an application speedup.

Our contributions are threefold. First, branch traces show that all 648 original invocations preserve prepared inputs, and per-batch engine counters quantify the removed requests and tokens. Second, three measurement scopes separate non-handler cost, time to the last search completion, and full-response time, each with an original-versus-original null control. The intermediate endpoint shows a repeatable subsecond reduction that the null control does not show, while full-response means do not resolve it. Third, source-set agreement is compared within and between application instances: two instances of the same arm differ as much as different arms, and identical inputs yield identical retrieval in only 65–79% of cross-instance cases, so between-arm quality gaps are not attributable to dispatch. Fourth, a rotated load sweep at concurrency 8–100 locates where the removed call matters: its own cost grows to tens of seconds per branch and tool-engine queueing falls, but the orchestrator engine saturates from concurrency 32 and bounds the end-to-end gain to a few percent. Input preservation alone does not establish equal final answers.

Our subject is this measurement gap, not a new security mechanism. The roughly eightfold local-to-response point-estimate ratio is cohort-sensitive, not universal.

2. Related Work

RAG supports generation with retrieved documents[1]. ReAct interleaves reasoning and actions[2]; Toolformer learns API use[3]; ToolLLM studies large tool collections[4]. Our intervention follows preparation rather than changing tool selection.

AutoGen structures agent conversations[5]. LLMCompiler separates planning and execution to parallelize function calls[6]. We remove one rewrite at a fixed handoff, without a new planner or scheduler.

Parrot and Agentix use application context for serving and scheduling[7], [8]. SGLang reuses KV state and supports structured decoding[9]; RAGCache caches knowledge states[10]. We instead test whether a call is needed, leaving the engine unchanged. Their workloads are not numerical baselines here.

HMAC authenticates messages with a shared se-

cret[11]; Macaroons extend it with contextual caveats[12]. Our local envelope is only a safeguard, not a new authorization design.

3. Existing Search Workflow and the Rewrite Step

The system determines the country and task, prepares terms and keywords, retrieves laws, selects evidence, and writes a response. For country comparisons it can launch two searches and pass selected evidence to a separate comparison worker. Figure 1 places the dispatch boundary within this workflow; Section 5 lists model roles.

The dispatch function receives country, merged keywords, search terms, and request ID. Only the first three form the tool argument object; merged keywords become its keywords field. The request ID is separate tracing context. A signed envelope uses a fresh UUID for scope, not that request ID.

The original route gives a model one registered search tool, then runs its returned call. Candidates copy the prepared object, check its shape, and invoke that handler directly. Eligibility is limited to this read-only operation with complete inputs. Invalid input selects the original route before issue. A failed envelope check after issue raises an error; it does not silently retry through a model.

The rewrite is removable work, not the main system bottleneck. Unique branch IDs link prepared and executed hashes, including concurrent country searches. The downstream comparison worker remains unchanged.

4. Dispatch Intervention and Measurement Scopes

4.1 A fixed execution boundary

An application adapter provides a controlled intervention after preparation. It registers a read-only operation, a validator, and an asynchronous handler. Unsigned dispatch copies the object through JSON serialization, checks its shape, and calls the existing handler. The validator requires exactly three fields: nonempty search terms, string or null country, and a list of string keywords. Request ID remains tracing context outside the payload. These checks do not judge relevance.

The signed control adds HMAC-SHA256, receiver, scope, expiry, and one-use nonce checks. Issuer and executor share a fresh key in one trusted process. There is no untrusted network hop; checked unsigned dispatch is sufficient here. The envelope is an optional deployment safeguard, not a security contribution or a speed mechanism. Distributed keys, replay storage, and attack resistance are outside this experiment.

Both candidates use the existing result parser and return fields. The original search loop permits one model round, retains the raw search-tool result in its trace, and passes that result to the parser. In the frozen application image, this route does not send the handler result to another model or sort or truncate the returned law list at this boundary. This source-level check addresses control flow; without full

output hashes it does not prove equal runtime outputs.

Eligibility requires complete inputs and a fixed operation. Tool choice, missing-value recovery, and argument correction remain model tasks where needed. Other domains require their own boundary checks.

4.2 Local work and request-level endpoints

We distinguish three scopes. The local boundary starts after preparation and ends after parsing the handler result. Its non-handler remainder is total boundary time minus nested handler time. The original remainder includes rewriting, communication, waiting, and common local work. Candidate measurements include copying, checks, dispatch, parsing, and tracing. This comparison is broader than signing cost alone.

For request i , let t_{0i} be client start, t_{pi} the first prepared-search event, t_{li} the last search-boundary completion, and T_i full response duration. For requests that use search, define

$$E_i = t_{li} - t_{0i}, \quad S_i = t_{li} - t_{pi}, \quad A_i = T_i - E_i. \quad (1)$$

E_i is time to the last search completion; S_i spans the search branches. A_i is the remaining response interval, including later selection, generation, communication, and waiting. These are chronological intervals, not additive model-compute times. Some work may overlap them.

Using the last branch rather than summing branches prevents double counting parallel country searches. E_i retains pre-search variation; S_i narrows attention to the search interval; T_i answers the user-visible question. Requests that bypass the boundary are excluded from intermediate endpoints but retained in full-response analysis.

Endpoints use existing client and application timestamps. All trial containers ran on the same host. Client wall-clock duration differed from its measured duration by at most 1.15 ms. This check supports timestamp consistency but does not remove logging overhead. Completion events are emitted after boundary work, so endpoints are observed completion times rather than exact GPU timestamps. Full parent-child spans would be needed to reconstruct every overlapping operation.

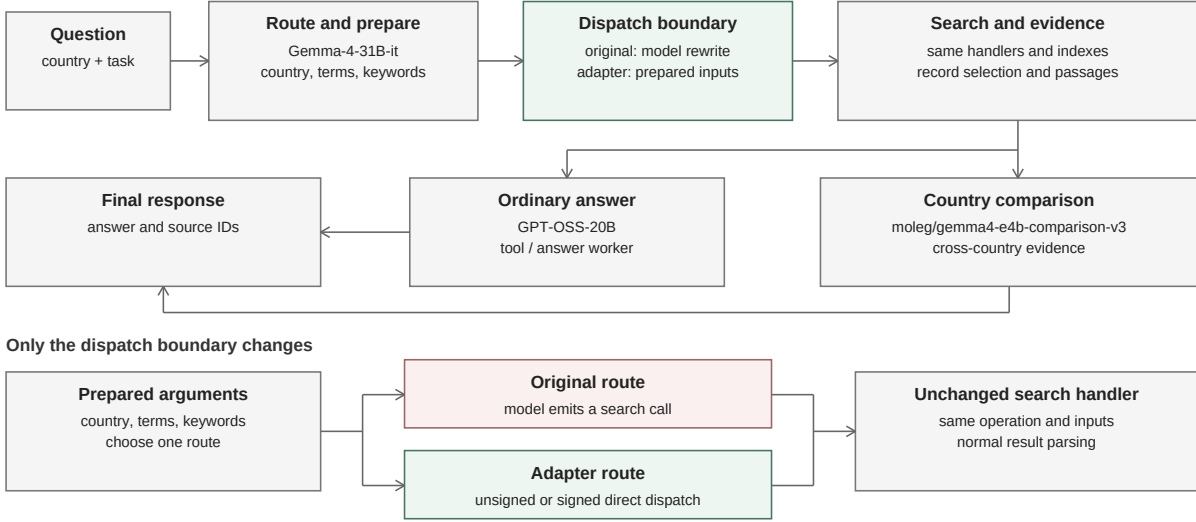
5. Experimental Method

5.1 System under test

An isolated legal search application used existing model and data services. Table 1 distinguishes orchestration, tool/answer, and country-comparison roles.

Model names are served IDs, not inferred training checkpoints. Two NVIDIA H200 NVL GPUs served all conditions with unchanged models, prompts, data, and vLLM options.

Application instances had two CPU cores and 8 GiB each; a separate client had two cores and 2 GiB. Thus the original app did not uniquely bear client cost. Model caches

Search workflow and the changed dispatch step

Envelope checks stay inside one trusted process; they are safeguards, not inference acceleration.

Existing safety service: Kanana-safeguard-8b. It is unchanged and outside the replaced step.

Fig. 1 Multi-agent search and the changed dispatch boundary. Both candidate routes call the same handler. Signing adds local checks, not inference acceleration.

Table 1 Model roles in the tested search application.

Component and model	Work in the search workflow
Orchestrator: Gemma-4-31B-it	Routing, preparation, selection, and fallback calls
Tool / answer: GPT-OSS-20B	Search-call rewriting, tool loops, and ordinary answers
Country comparison: moleg/gemma4-e4b-comparison-v3	Writes comparisons from country-specific evidence
Safety: Kanana-safeguard-8b	Existing request safety checks
Search handlers: text and title indexes	Records for selection and evidence preparation

Table 2 Indexed records shared by all three conditions.

Index	Records
Original-law chunks	3,194,266
Translated-law chunks	454,351
Law-title records	19,583
Total	3,668,200

and external service traffic remained shared.

These are index records, not unique laws or articles. One law can occupy multiple chunks and language indexes.

5.2 Questions, conditions, and execution

The frozen set contained 200 questions across 20 countries, with a country name in every question. Four types had 50 questions each: finding a law, explaining content, compar-

ing countries, and analyzing a case. All nine batches used identical question IDs and text, checked after collection. A comparison question counted once even if it triggered two country searches.

Null country preserves the optional-country interface: a country named in a question need not resolve to a country field. The adapter does not infer or repair this value.

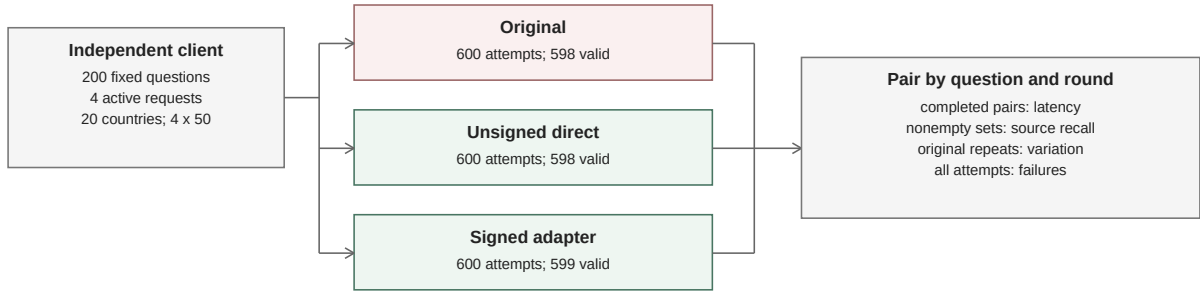
All conditions used one application image, schema, parser, and branch tracing. Only prepared-input dispatch changed: original model rewriting, checked direct execution, or signed direct execution. Each condition ran in its own application instance, which stayed up for all three rounds; arm and instance are therefore confounded.

Each condition processed 200 questions in each of three rounds, for 600 requests per condition and 1,800 overall. The serial order was original–unsigned–signed in round 1, unsigned–signed–original in round 2, and signed–original–unsigned in round 3. This is a cyclic 3×3 Latin-square layout: each method occupies each round and position once[13]. The rotation was fixed, not randomly selected. Equal-weight method means balance additive round and position effects when all cells use the same questions. They do not remove interactions or cache carryover. One smoke request per condition was excluded.

The driver kept four requests active, submitting another question when one finished. It used the same seeded question order in every batch. This closed-loop design measures four active clients, not an open arrival process and not a high-user-count deployment. Application admission behavior was unchanged.

The client used a 180-second read/write/pool timeout

Repeated evaluation: separate client, same questions



Round 1: original / unsigned / signed. Round 2: unsigned / signed / original.

Round 3: signed / original / unsigned. Each condition occupies each serial position once.

1,800 attempts; 1,795 valid responses. The same 26 questions bypass the boundary in every batch.

Models, prompts, serving settings, search indexes, and application resources remain unchanged.

Fig. 2 Completed three-condition evaluation. Each method receives the same 200 questions per round. Failed requests are reported separately from completed-pair latency and source agreement.

and a 15-second connection timeout. The 180-second setting is not a hard cap on total request time: earlier processing can precede a stalled read. An incomplete request duration is therefore not the latency of a complete answer. Results retain failures rather than replacing them with fast empty responses.

Request time began after the client acquired a concurrency slot, immediately before the HTTP request, and ended at the final response or error. It excludes client-slot waiting. Batch wall time covers the complete batch. Valid-response throughput is valid responses divided by batch wall time; at fixed concurrency it is closely related to latency and is not independent evidence of acceleration.

The runner checked availability, idle state, telemetry, errors, and progress. All batches completed; shared background load was not controlled.

5.3 Branch traces and agreement metrics

Unique asynchronous branch IDs link prepared and handler-entry hashes under the same JSON rules. Traces record changed field names but not raw arguments. Parallel country branches remain distinct.

Boundary timing excludes the prepared-input trace and includes result parsing. A nested timer covers the handler. Separate signed intervals measure setup, issue, and verification through handler entry; unsigned intervals measure copying/validation and entry. Entry intervals include hash observation, not pure cryptographic work.

Request IDs and timestamp intervals joined logs to all 200 client records per batch, distinguishing repeated case IDs. Smoke traces were excluded. Every prepared branch had one handler and one completion record.

Time comparisons pair the same question within a round and retain only responses completed by both methods. A valid response requires successful client status, a law list, and an answer string. We report all-attempt durations

and failures separately. Different pairwise comparisons may exclude different questions; their denominators must not be silently merged.

Let B_i and C_i be the deduplicated baseline and candidate law-ID sets and $h_i = |B_i \cap C_i|$. When both sets are nonempty,

$$P_i = \frac{h_i}{|C_i|}, \quad R_i = \frac{h_i}{|B_i|}, \quad (2)$$

and

$$F_i = \begin{cases} 2P_i R_i / (P_i + R_i), & P_i + R_i > 0, \\ 0, & P_i + R_i = 0. \end{cases} \quad (3)$$

The frozen evaluator assigns all three scores 1 when both sets are empty and 0 when only one is empty. Macro scores average the question-level values, including the per-question F1 rather than the harmonic mean of macro precision and recall. We emphasize nonempty-reference scores and report empty counts explicitly.

Candidates use the same-round original as reference; original-repeat comparisons use the earlier round. These are source-ID agreement scores, not expert legal accuracy.

For each completed pair, $d_i = T_{B,i} - T_{C,i}$; positive values favor the candidate. We report mean and median differences. Within each round, 10,000 question-pair resamples with seed 20260916 give a descriptive percentile interval for mean saving. An exact two-sided sign test compares positive and negative differences, excluding ties. Concurrent questions share service state, so these are exploratory within-round summaries, not independent trials of a deployment-wide effect. Multiple comparisons are not adjusted.

Overall summaries give equal weight to round-specific paired means, using unrounded values. They are not pooled significance tests. A common-question summary retains only questions completed in all nine batches to preserve exact question, round, and position balance.

Null controls apply the same paired machinery to original-versus-original pairs across rounds and to unsigned-versus-signed pairs within rounds. These pairs share the mechanism, so their differences estimate ordinary variation. We also report a 10% trimmed mean of paired differences and a geometric-mean ratio from mean log ratios; both limit the influence of a few very long responses. The position-balanced method means receive a question bootstrap that resamples the 195 common questions with all nine cells attached.

Engine load is read from vLLM counters sampled during each batch. For each served model, the difference between the last and first sample gives completed requests, prompt and generation tokens, and summed queue time. Wrapped application calls provide per-call usage; the remainder is native, unwrapped traffic such as the search rewrite loop. Requests on the orchestrator and safety engines are compared with wrapped counts to detect outside traffic.

Source-set agreement is computed within an arm across rounds (self-repeat) and between arms within a round. For each pair of arms, we count questions whose law set is identical in all three rounds of both arms yet differs between the arms. Such stable differences cannot come from per-request sampling noise.

A follow-up run of the same harness used two instances of the original path and two of signed dispatch, each serving the 200 questions once at concurrency four in the order original-a, signed-a, original-b, signed-b (800 requests). Its attachment additionally records, per branch, SHA-256 hashes of the handler’s raw result, of the returned law-ID list, and of the object passed to the next stage, so that instances can be compared on identical prepared inputs.

For planning sensitivity, let s_d be the observed SD of paired full-response differences. For target difference δ , a normal approximation gives

$$n \approx [(z_{0.975} + z_{0.8})s_d/\delta]^2. \quad (4)$$

This assumes a two-sided 5% test, 80% power, and independent stable differences[14]. We set $\delta = 0.586$ seconds as a local-work-scale target, not an assumed true response gain. Estimated variance, heavy tails, and shared queues make this a planning sensitivity, not a guaranteed sample size.

5.4 Load sweeps

Two sweeps compared the original path with signed dispatch under load on the same frozen image, question set, and seed, with the isolated instance’s admission limit raised from 4 to 100 so that client concurrency reached the engines, and a 600-second request timeout. An exploratory single pass at concurrency 1, 2, 5, 10, 20, 50, and 100 ran one batch per arm per level with alternating order; it is summarized in one paragraph of Section 6.5 because arm and serial position are confounded within each level.

The confirmatory sweep ran concurrency 8, 16, 32, 64, and 100 with two rounds per level, the arm order reversed

between rounds (original first, then signed first). Each cell is one closed-loop 200-question batch; the sweep contains 20 cells and 4,000 requests, all of which completed without transport failure. Cells carried the branch traces of Section 5.3, so the rewrite call’s own duration under load is observed directly as the original arm’s non-handler time, and vLLM counters were sampled every second. The high-load safety policy stopped a batch on telemetry loss, an engine queue above 256 for 30 seconds, four failures among the last twenty responses, or no progress. Before each cell the runner waited for idle engines. After the tenth cell, one request that never completed remained running in the orchestrator engine and the strict gate could not be satisfied; the remaining ten cells used a gate that requires zero waiting requests and tolerates one running request per engine, and both arms ran under this condition. The same gate and the same stuck request applied to the follow-up run. Production traffic shared the engines throughout.

Per level, we pair each question within a round, average its two round differences, and report the mean of these per-question values with a question bootstrap interval, a sign test on their signs, and each round separately. Throughput, the share of requests within 60 seconds, engine counters per request, and within-arm agreement between rounds are reported per level.

6. Results

All 1,800 attempts completed. There were 1,795 valid responses and five read timeouts: two in the original condition, two in unsigned dispatch, and one in signed dispatch. Question IDs and text match across all conditions and rounds.

6.1 Response time and repeatability

Table 3 gives completed-pair latency. Signed dispatch was slower in round 1 and faster in rounds 2 and 3. Its paired mean changes were -5.42% , 0.80% , and 5.25% , respectively. Unsigned dispatch also changed direction, at -8.52% , 1.00% , and 4.94% . Thus neither candidate consistently improved the round mean.

Equal-weight round means were 24.505 seconds for the paired original and 24.434 seconds for signed dispatch: a 0.071-second, 0.29% decrease. For unsigned dispatch, corresponding means were 24.530 and 24.714 seconds: a 0.184-second, 0.75% increase. Baseline means differ because the two pairwise comparisons exclude different failed questions. These summaries describe the recorded rounds; their uncertainty is too large to resolve a subsecond mean effect.

Signed dispatch had positive median savings of 0.022, 0.226, and 0.556 seconds. Faster/slower counts were 100/99, 110/90, and 117/81, with no ties. Exact two-sided sign-test values were 1.000, 0.179, and 0.0127. Only the third round gives a strong directional signal under this exploratory test, while all three mean intervals include zero. A positive median can coexist with a negative mean when a smaller

Table 3 Completed-pair response time by round. Positive saving means faster candidate responses. Intervals resample questions within each round; they do not measure between-round uncertainty.

Round	Candidate	Pairs	Original (s)	Candidate (s)	Saving (s)	Change (%)	95% interval (s)
1	Unsigned direct	198	23.733	25.755	-2.022	-8.52	[-5.364, 0.773]
1	Signed adapter	199	23.677	24.960	-1.283	-5.42	[-3.145, 0.354]
2	Unsigned direct	200	25.151	24.901	0.251	1.00	[-2.076, 2.308]
2	Signed adapter	200	25.151	24.950	0.201	0.80	[-2.242, 2.061]
3	Unsigned direct	198	24.706	23.487	1.220	4.94	[-0.662, 3.111]
3	Signed adapter	198	24.687	23.391	1.296	5.25	[-0.543, 3.288]

number of large delays dominate the average.

Unsigned median savings were 0.267, 1.160, and 0.601 seconds; sign-test values were 0.286, 0.000164, and 0.00544. Directional gains can coexist with unresolved mean gains because the sign test ignores difference size.

Table 4 All-attempt sensitivity over 600 requests per condition. Throughput uses total valid responses divided by summed batch wall time.

Metric	Original	Unsigned	Signed
Mean duration (s)	25.041	25.221	24.725
Valid responses	598	598	599
Read timeouts	2	2	1
Responses/s	0.1555	0.1568	0.1596

All-attempt signed duration is 1.26% lower versus 0.29% in paired means; Table 4 is therefore secondary. Failures span 186.848–191.643 seconds: content explanation in original/unsigned round 1; case analysis in original/signed round 3; content explanation in unsigned round 3. Failed questions differ between methods.

Signed p95 durations were 55.096, 53.001, and 56.043 seconds versus original 54.074, 53.254, and 56.787 seconds: small, mixed changes. One fewer timeout is insufficient evidence of greater reliability.

Signed versus unsigned saving was 0.729 seconds in round 1, then -0.049 and -0.043 seconds. These differences cannot be explained by submillisecond signing overhead.

Table 5 places the candidate comparisons beside two null controls. Original rounds differ from each other by -1.526, -1.019, and 0.546 seconds, with paired SDs of 10.9–13.8 seconds. These differences are as large as the candidate effects and also change sign. Trimmed means and geometric ratios narrow the intervals without changing the picture: unsigned dispatch has positive trimmed savings that exclude zero in rounds 2 and 3, signed dispatch does not, and every null-control interval includes zero. Unsigned-versus-signed differences are of the same size as candidate-versus-original differences.

On the common 195-question cohort, signed dispatch saves 0.123 seconds (0.50%) with a question-bootstrap interval of [-1.153, 1.290] seconds; unsigned dispatch adds 0.077 seconds, interval [-1.465, 1.952]. Averaging each question over its three rounds, unsigned dispatch is faster on 114 of 195 questions (median 0.625 seconds, sign $p = 0.022$) and signed dispatch on 100 (median 0.065

seconds, $p = 0.775$). Geometric-mean changes are 2.1% [-1.4, 5.4] and 1.5% [-1.6, 4.5]. These means are balanced for additive round and position effects. Table 6 shows a 1.130-second position range, larger than the 0.586-second local-work target, and original round means move from about 23.7 to 25.2 seconds, roughly 6%. Position variation is consistent with the sign changes, but one nonrandomized square cannot separate carryover and interactions reliably.

At this target, normal-approximation power is only 8.4–10.1% in signed comparisons. About 3,600–5,300 independent pairs would be needed, roughly 19–27 batches of 200 pairs. Repeating the same questions that many times is not automatically equivalent to independent observations. This design is underpowered for a subsecond full-response reduction; failure to detect one is not evidence that removal has no benefit.

6.2 Intermediate completion times

Table 8 pairs completed requests that use the search boundary in both conditions. Signed dispatch reduces request-start-to-last-search time by 0.529, 0.639, and 0.464 seconds. All three within-round descriptive intervals are above zero. First-search-to-last-search savings are 0.543, 0.504, and 0.561 seconds, with narrower intervals: [0.511, 0.577], [0.483, 0.526], and [0.522, 0.606]. This places a repeatable reduction within the search interval.

For signed dispatch, paired SD of E is 2.17–2.42 seconds, while that of the post-search remainder is 13.04–16.10 seconds. Post-search savings are -2.017, -0.367, and 1.019 seconds. On these same pairs, full-response savings are -1.488, 0.272, and 1.483 seconds. This decomposition locates larger variation after search; it does not show that call removal causes later slowing.

Unsigned first-search-to-last-search savings are 0.523, 0.497, and 0.550 seconds, with all three descriptive intervals above zero. Its broader request-start endpoint in round 3 includes zero because preparation also varies. The same endpoints between original rounds change by 0.035, -0.023, and -0.058 seconds (first to last search) and by -0.120, -0.068, and 0.052 seconds (request start to last search), all within 0.12 seconds. The candidate reductions therefore exceed ordinary variation at this scope by an order of magnitude. Signed search-interval savings by type are 0.42–0.47 seconds for law finding, 0.48–0.52 for content explanation, 0.55–0.64 for country comparison, and 0.57–0.70 for case

Table 5 Null controls and robust summaries for completed pairs. Saving is the first member minus the second. The trimmed mean removes 10% of differences at each end; geometric change is one minus the geometric mean of candidate/original ratios. Intervals are question-bootstrap percentiles.

Round(s)	Pair	Pairs	Mean saving (s)	Trimmed mean (s) [95%]	Geometric change (%) [95%]	Faster/slower	Sign p
1, 2	Original vs original	199	-1.526	-0.393 [-1.198, 0.438]	-3.51 [-9.14, 1.90]	95/104	0.571
1, 3	Original vs original	198	-1.019	-0.165 [-1.020, 0.649]	-2.84 [-8.54, 2.31]	95/103	0.619
2, 3	Original vs original	199	0.546	0.179 [-0.493, 0.931]	0.85 [-3.93, 5.50]	102/97	0.777
1	Original vs unsigned	198	-2.022	0.392 [-0.655, 1.391]	-1.32 [-7.98, 4.66]	107/91	0.286
2	Original vs unsigned	200	0.251	1.251 [0.316, 2.178]	3.44 [-2.33, 8.86]	127/73	0.0002
3	Original vs unsigned	198	1.220	0.942 [0.089, 1.934]	3.17 [-2.24, 8.16]	119/79	0.0054
1	Original vs signed	199	-1.283	-0.380 [-1.345, 0.485]	-2.53 [-7.55, 2.17]	100/99	1.000
2	Original vs signed	200	0.201	0.627 [-0.233, 1.553]	1.70 [-3.50, 6.58]	110/90	0.179
3	Original vs signed	198	1.296	0.596 [-0.135, 1.421]	4.60 [-0.32, 9.41]	117/81	0.0127
1	Unsigned vs signed	199	0.729	-0.523 [-1.471, 0.480]	-1.20 [-7.27, 4.83]	90/109	0.202
2	Unsigned vs signed	200	-0.049	-0.554 [-1.360, 0.310]	-1.81 [-7.75, 3.80]	91/109	0.229
3	Unsigned vs signed	198	-0.043	-0.137 [-0.881, 0.608]	1.01 [-3.36, 5.32]	102/96	0.722

Table 6 Latin-square means on the same 195 questions completed in all nine batches. Each entry averages three cell means.

Position	First	Second	Third
Mean (s)	24.086	25.216	24.665
Method	Original	Unsigned	Signed
Mean (s)	24.671	24.748	24.548

Table 7 Full-response planning sensitivity for a 0.586-second target. MDE is the detectable effect at 80% power under the stated approximation.

Candidate	Round	SD (s)	MDE (s)	Pairs needed
Signed	1	12.562	2.495	3,607
Signed	2	15.261	3.023	5,324
Signed	3	13.556	2.699	4,201
Unsigned	1	22.185	4.417	11,250
Unsigned	2	15.918	3.153	5,792
Unsigned	3	13.385	2.665	4,095

analysis. These exploratory endpoints from saved traces support local timing benefits without establishing full-response acceleration. They were not prespecified confirmatory tests.

6.3 Source agreement and question types

Table 9 excludes empty reference sets. Signed recall was 98.25–98.53% across rounds. Repeated original runs gave 98.62–99.13%, showing that the original system also varies, but with somewhat higher agreement in these comparisons. There is no prespecified non-inferiority margin or expert reference; we therefore do not claim unchanged answer accuracy.

Each condition in each round produced the same 26 valid empty law-ID sets. Joining IDs to branch traces shows that these were exactly the questions that did not call the measured search boundary: four law-finding, seven content-explanation, eight country-comparison, and seven case-analysis questions. There were no one-sided empty sets in the reported paired comparisons. The empty outputs did not come from an empty handler result; the workflow bypassed that handler.

Including empty pairs raises signed macro recall to 98.72%, 98.48%, and 98.69%. Main quality results instead use 173, 174, and 172 nonempty references, excluding failed pairs.

Exact law-set matches were 182/199, 184/200, and 183/198 for signed dispatch and 175/198, 182/200, and 173/198 for unsigned dispatch, including empty pairs. Repeated original runs matched in 190/199, 187/198, and 192/199 pairs. On the 195 common questions, between-arm agreement is below original-repeat agreement by 3.6 percentage points [0.2, 7.2] for signed and 6.2 points [2.9, 9.7] for unsigned dispatch; the nonempty-reference recall gaps are 0.5 points [-0.6, 1.7] and 1.6 points [0.5, 2.8] over 169 questions.

Table 10 separates two kinds of agreement. Each arm agrees with itself across rounds at 95.2–95.4%, so the candidates are as repeatable as the original path. Any two different arms agree at only 89.2–91.8% within a round, and the two direct arms, which share one mechanism, agree at 91.1%. Between 173 and 175 questions have an identical set in all three rounds of both arms of a pair; among these, 10 (original versus unsigned), 8 (original versus signed), and 6 (unsigned versus signed) differ between the arms. Stable differences of this kind cannot arise from per-request sampling noise. Each arm ran in its own application instance, so every between-arm comparison is also a between-instance comparison. Instance-level state, such as hash-seed-dependent ordering or instance-local caches, is a sufficient explanation, and the design cannot attribute the between-arm gap to dispatch. Country comparison shows the lowest agreement under every comparison: 88.0% original self-repeat, 82.0% original versus signed, and 75.3% original versus unsigned.

Within-branch preservation is not cross-run input equality. Original and signed prepared-hash multisets match in 170/173, 169/174, and 167/172 boundary-used completed pairs; original and unsigned match in 166/172, 170/174, and 165/172. The three-round study did not record handler-output hashes; the follow-up run did. Table 11 gives its result. Two instances of the same arm agree on exact law sets in 86.4% (original) and 83.5% (signed) of questions, no

Table 8 Intermediate endpoints on boundary-used completed pairs within each row. Positive saving favors the candidate. Post-search is the remainder after the last search completion, not pure generation time.

Round	Candidate	Pairs	Original E (s)	Candidate E (s)	Saving E (s)	95% interval (s)	Saving A (s)
1	Unsigned	172	7.741	7.257	0.484	[0.140, 0.830]	-2.857
1	Signed	173	7.727	7.198	0.529	[0.211, 0.851]	-2.017
2	Unsigned	174	7.832	7.061	0.771	[0.459, 1.086]	-0.454
2	Signed	174	7.832	7.193	0.639	[0.297, 1.000]	-0.367
3	Unsigned	172	7.793	7.478	0.315	[-0.053, 0.684]	1.064
3	Signed	172	7.799	7.335	0.464	[0.115, 0.823]	1.019

Table 9 Nonempty-reference source agreement. Precision, recall, and F1 are macro percentages. Original-repeat rows use the earlier round as reference; they are not additional candidate trials.

Reference	Compared result	Completed pairs	Nonempty references	Precision	Recall	F1
Original, round 1	Unsigned, round 1	198	172	97.38	97.61	96.98
Original, round 2	Unsigned, round 2	200	174	98.27	97.63	97.59
Original, round 3	Unsigned, round 3	198	172	96.88	96.83	96.25
Original, round 1	Signed, round 1	199	173	98.25	98.53	98.03
Original, round 2	Signed, round 2	200	174	98.31	98.25	97.88
Original, round 3	Signed, round 3	198	172	98.61	98.50	98.22
Original, round 1	Original, round 2	199	173	99.18	98.87	98.77
Original, round 1	Original, round 3	198	172	98.98	98.62	98.50
Original, round 2	Original, round 3	199	173	99.18	99.13	98.90

Table 10 Exact law-set agreement on the 195 common questions. Self-repeat compares an arm with itself across rounds (pairs 1-2, 1-3, 2-3); between-arm rows compare arms within rounds 1, 2, and 3.

Comparison	Mean (%)	Pair 1	Pair 2	Pair 3
Original self-repeat	95.4	95.4	94.4	96.4
Unsigned self-repeat	95.2	96.9	94.4	94.4
Signed self-repeat	95.4	94.9	95.4	95.9
Original vs unsigned	89.2	88.7	91.3	87.7
Original vs signed	91.8	91.3	91.8	92.3
Unsigned vs signed	91.1	92.8	89.7	90.8

more than any two instances of different arms (83.0–87.0%). Prepared arguments were identical across instances for 164–170 of 174 boundary-using questions, but for those identical inputs the search handler returned an identical law-ID list in only 65–79% of questions and a byte-identical result in 5–17%. The retrieval path is therefore nondeterministic across instances, and the between-arm gaps of the three-round study lie within this instance-level variation. Paired latency between the two instances of one arm differed by -0.6 and +0.4 seconds, and between arms by 0.2–1.1 seconds in favor of signed dispatch, all with intervals including zero.

Case analysis had the lowest recall in each round. This type requires combining evidence into a scenario, so missing a reference law is important even if the overall mean remains high. We do not attribute the difference to signing or to one particular downstream model without a source-selection trace comparison.

Type-level mean savings also change direction across rounds. The largest changes do not consistently occur in country comparisons, the group using two search branches.

Table 11 Follow-up with two instances per arm at concurrency four (200 questions each): exact law-set agreement, nonempty-reference recall, and, among questions whose prepared arguments were identical in both instances, the number with identical law-ID lists from the search handler.

Pair	Kind	Exact set (%)	Recall (%)	Same law list
orig-a / orig-b	same arm	86.4	96.8	111/164
signed-a / signed-b	same arm	83.5	97.0	114/170
orig-a / signed-a	different	86.4	97.0	126/166
orig-b / signed-b	different	87.0	96.7	132/168
orig-a / signed-b	different	83.9	95.6	117/168
orig-b / signed-a	different	83.0	96.5	109/168

Table 12 Signed-adaptor recall by question type, using nonempty references. Parentheses give the reference count.

Type	Round 1	Round 2	Round 3
Find law	100.00 (46)	100.00 (46)	98.91 (46)
Explain content	100.00 (42)	100.00 (43)	100.00 (43)
Compare countries	97.79 (42)	98.27 (42)	97.68 (42)
Analyze a case	96.25 (43)	94.63 (43)	97.29 (41)

6.4 Removed work and remaining delay

Each batch contains 216 search branches: 132 questions use one, 42 country comparisons use two, and 26 bypass search. All branches complete without schema fallback or boundary error, including requests that later time out.

Prepared/executed hashes match in all 648 branches of each condition. Unique branch IDs separate concurrent searches. This establishes input preservation for recorded branches, not every possible question.

Candidates bypass 648 rewrite opportunities each.

Other model-based stages can change their call counts, so this is not a whole-system net call reduction.

Table 13 Live boundary timing, pooled over 648 branches per condition. Values are milliseconds per branch. Handler time is measured inside total time.

Interval	Original	Unsigned	Signed
Full boundary	1118.451	588.066	578.052
Search handler	575.311	587.545	577.173
Non-handler remainder	543.140	0.521	0.880
Setup / key creation	–	–	0.083
Issue	–	–	0.249
Copy and validate	–	0.070	–
Verify / handler entry	–	0.052	0.144
Unitemized residual	–	0.399	0.404

Table 13 compares full non-handler intervals, not pure inference or cryptography. The residual row is non-handler time minus listed subintervals, including unitemized parsing and tracing. Rounding can leave 0.001 ms differences. All intervals exclude prepared-input tracing before the boundary timer.

Original non-handler round means were 544.393, 513.765, and 571.261 ms; both candidates remained below 1 ms. Signed overhead exceeded unsigned overhead by 0.358 ms per branch.

At 216 branches per 200 questions, the non-handler difference corresponds to approximately

$$(543.140 - 0.880) \frac{216}{200} \frac{1}{1000} = 0.586 \text{ s} \quad (5)$$

of removed branch work per attempted question. This is about 2.4% of a 24.5-second completed response. It is not an expected or guaranteed response-time saving: two country branches can overlap, later work varies, and queue interactions can change across runs.

The ratio $0.586/0.071 \approx 8.2$ shows that treating branch work as response saving would give about eight times the paired point estimate. This is not a stable factor: the identical 195-question balanced cohort gives $0.586/0.123 \approx 4.8$, and mean-saving intervals include zero. Work and response time have different scopes. Intermediate endpoints provide stronger evidence: a subsecond search reduction is present, while larger later variation limits full-response detection.

Wrapped application calls place the remaining delay in context. In the original runs, pooled mean API intervals were 3.597 seconds for orchestrator calls, 6.760 seconds for ordinary answer synthesis, and 9.570 seconds for country comparison. These include API waiting and communication, not only decoding. They are per-call means, not additive shares of one answer. Answer-synthesis means varied from 5.776 to 7.553 seconds across original rounds, much more than the local envelope overhead.

Each candidate removes 648 search-rewrite invocations over 600 attempted questions, or 1.08 per attempt. Table 14 reads the tool/answer engine’s counters per batch.

The original path completes 500 requests per batch on that engine, of which 139 are wrapped answer-synthesis calls and 361 are native tool-loop requests; both candidates complete 293–294 requests, 41% fewer, with 153–154 native requests remaining. The per-batch prompt-token difference is 21,500–34,100 tokens, or 1.4–2.3% of the engine’s prompt load, about 105–165 prompt tokens per removed request. Generation-token differences (–1,500 to +30,500 per batch) lie within answer-length variation, and summed queue time falls from 9.2 seconds to below 0.01 seconds. The rewrite is request-heavy but token-light: its removal reduces request count far more than served tokens.

Table 14 Tool/answer engine counters per batch, mean of three rounds. Native requests are counter requests minus wrapped application calls.

Per batch	Original	Unsigned	Signed
Requests	500.0	293.3	294.0
Wrapped calls	139.0	140.7	140.0
Native requests	361.0	152.7	154.0
Prompt tokens	1,509,602	1,475,537	1,488,060
Generation tokens	365,919	367,427	335,424
Summed queue time (s)	9.23	< 0.01	< 0.01
Summed request latency (s)	1,677	1,665	1,514

Orchestrator-engine request counts equal wrapped orchestrator calls within one request in all nine batches, the safety engine served exactly 200 requests per batch, and the comparison engine counted 35 of 42 wrapped calls, the remainder being input-length rejections. No outside traffic reached these engines during the batches. Local envelope contract checks are implementation tests, separate from live latency and quality results.

6.5 Load dependence

The exploratory single pass had shown a 3.18-second (5.4%) paired saving at concurrency 20 with an interval above zero, but sign changes at 50 and 100 (–9.55 and +7.71 seconds) in which the arm that ran first was faster. The rotated sweep replaces it. Table 15 gives its paired latency, Table 16 the throughput and boundary measurements, and Table 17 the engine counters.

The removed call is far more expensive under load than at concurrency 4. The original arm’s rewrite time per branch grows from 1.2 seconds at concurrency 8 to 4.5, 6.4, 19.1, and 22.4 seconds at 16, 32, 64, and 100, with p95 values of 16–32 seconds from concurrency 16 upward, because the rewrite waits in the tool engine’s queue. Signed dispatch keeps its non-handler time at 1.2–1.6 ms. Correspondingly, the tool engine serves 41–46% fewer requests per batch and its queue time per request falls by 29–72% (from 1.19 to 0.63 seconds at 8, from 39.0 to 10.8 seconds at 100), while its maximum waiting depth falls from 17–143 to 9–21 requests.

The end-to-end saving is much smaller than the removed call. Pooled savings are 0.44 seconds (1.4%) at 8, 1.54 (2.9%) at 16, –3.35 (–3.6%) at 32, 3.54 (2.6%) at

Table 15 Rotated load sweep: completed-pair response time, original versus signed dispatch, 200 pairs per cell. Saving is original minus signed. Pooled rows average each question's two round differences; their intervals are question-bootstrap percentiles and their sign tests use the per-question averages.

C	Round (first arm)	Original (s)	Signed (s)	Saving (s)	95% interval (s)	Median (s)	Faster/slower	Sign <i>p</i>
8	1 (original)	32.23	32.52	-0.29	[-3.41, 2.14]	0.81	117/83	0.019
8	2 (signed)	32.25	31.08	1.18	[-0.97, 3.21]	0.31	105/95	0.525
8	pooled	32.24	31.80	0.44 (1.4%)	[-1.44, 2.16]	-	117/83	0.019
16	1 (original)	52.81	52.18	0.63	[-2.48, 3.41]	2.85	121/79	0.004
16	2 (signed)	54.68	52.22	2.46	[-0.69, 5.64]	2.72	118/82	0.013
16	pooled	53.75	52.20	1.54 (2.9%)	[-0.97, 3.98]	-	120/80	0.006
32	1 (original)	90.13	98.11	-7.98	[-11.36, -4.69]	-3.42	66/134	< 0.001
32	2 (signed)	93.81	92.54	1.28	[-2.74, 5.22]	-0.14	99/101	0.944
32	pooled	91.97	95.32	-3.35 (-3.6%)	[-6.14, -0.67]	-	89/111	0.137
64	1 (original)	138.14	129.79	8.35	[3.89, 12.62]	8.03	138/62	< 0.001
64	2 (signed)	135.37	136.63	-1.26	[-7.29, 4.06]	3.09	123/77	0.001
64	pooled	136.75	133.21	3.54 (2.6%)	[-0.57, 7.46]	-	128/72	< 0.001
100	1 (original)	163.89	157.76	6.13	[2.00, 10.13]	5.66	149/51	< 0.001
100	2 (signed)	164.04	162.71	1.33	[-3.63, 6.02]	1.51	128/72	< 0.001
100	pooled	163.97	160.24	3.73 (2.3%)	[0.10, 7.20]	-	149/51	< 0.001

Table 16 Rotated load sweep: throughput, completion, and boundary measurements per level, mean of two rounds, original / signed. Rewrite time is the original arm's non-handler boundary time per branch; the signed arm's non-handler time is 1.2–1.6 ms at every level.

C	Valid responses/s	Within 60 s (%)	Rewrite mean / p95 (s)	Search handler (s)
8	0.243 / 0.245	93 / 92	1.2 / 4.3	0.61 / 0.62
16	0.286 / 0.292	63 / 65	4.5 / 16.2	0.57 / 0.68
32	0.301 / 0.306	15 / 18	6.4 / 22.0	0.69 / 0.84
64	0.395 / 0.377	10 / 10	19.1 / 32.0	2.80 / 3.27
100	0.469 / 0.506	9 / 9	22.4 / 29.0	5.00 / 5.97

Table 17 Rotated load sweep: engine counters per level, mean of two rounds, original / signed. Queue times are per-batch sums divided by 200 requests; waiting depth and KV usage are batch maxima. Worker is the tool/answer engine.

C	Worker requests	Worker queue/request (s)	Worker max waiting	Orch. queue/request (s)	Orch. max waiting	Orch. max KV
8	517 / 305	1.19 / 0.63	7 / 4	0.14 / 0.05	4 / 4	0.65 / 0.60
16	521 / 306	8.44 / 6.03	17 / 9	0.45 / 0.36	5 / 6	0.91 / 0.95
32	521 / 301	12.11 / 3.87	39 / 10	28.4 / 29.4	29 / 26	1.00 / 1.00
64	508 / 286	33.72 / 14.63	91 / 20	86.9 / 84.6	78 / 67	1.00 / 1.00
100	456 / 246	39.04 / 10.75	143 / 21	121.7 / 137.9	136 / 148	1.00 / 1.00

64, and 3.73 (2.3%) at 100. Per-question sign tests favor signed dispatch at 8, 16, 64, and 100 (up to 149 of 200 questions faster at 100), and both rounds at 16, 64, and 100 have positive medians; the pooled mean interval excludes zero only at 100. The cause is visible in Table 17: from concurrency 32 the orchestrator engine reaches 100% KV usage with preemptions, and its queue time per request rises from 0.1–0.5 seconds to 28, 85–87, and 122–138 seconds. Time removed from the search branch is largely spent waiting for the orchestrator instead. At 100 the signed arm even carries more orchestrator queueing than the original (138 versus 122 seconds per request; maximum waiting 148 versus 136), because its requests reach the orchestrator sooner. Concurrency 32, where saturation begins, is the least stable level: the two rounds disagree by 9 seconds and the pooled mean is negative although the per-question sign test is not.

Throughput and completion within 60 seconds are equal between arms at every level (93–94% within 60 seconds at 8, 61–66% at 16, 15–18% at 32, 8–11% at 64 and 100); the closed loop fixes throughput, and saturation, not dispatch, determines the tail. The search handler itself also slows from about 0.6 seconds to 2.8–3.3 seconds at 64 and 5.0–6.0 seconds at 100, showing that the retrieval services are shared under load as well.

Saturation degrades output stability for both arms. Within-arm exact law-set agreement between the two rounds is 92–94% for both arms up to concurrency 32, then 83% and 82% at 64 and 81% and 73% at 100 (original and signed respectively); within-arm nonempty recall falls from 97–99% to 84% and 80% at 100. Between-arm agreement within a round is 85–89% up to 32 and 74–82% above it. As in Section 6.3, one instance per arm prevents attributing the

between-arm gap to dispatch; the load effect is common to both arms.

7. Discussion

Input preservation and roughly half-second search-interval savings are supported, and original-versus-original controls at the same scope stay within 0.12 seconds. Full-response sensitivity is low at this scale: original rounds differ from each other by up to 1.5 seconds, more than the expected saving. A null label loses the intermediate finding; an end-to-end speedup claim exceeds the evidence.

A large relative reduction in a short boundary need not improve the whole application proportionally. The cohort-sensitive eightfold ratio illustrates scope mismatch, not a general correction factor. Linked endpoint measurements provide stronger evidence.

Unsigned dispatch already removes rewriting; signing adds 0.358 ms per branch. Much larger response differences are not explained by that cost. Trusted in-process execution needs no signing; distributed use needs its own trust-boundary argument.

Argument hashes establish that a branch executes what it prepared. They do not show that separate runs prepared the same query, retrieved the same ordered records, or passed equal objects to later agents. The frozen source has no extra post-handler model round or list sorting at this boundary, but full runtime output hashes remain missing. A stronger study should hash handler output and the next-stage object, preserving order, and compare pairs with equal upstream inputs before locating later differences.

The same 26 questions bypass search in every batch; agreed empty sets are not evidence of correct retrieval. Between-arm source-set agreement is lower than within-arm repeatability, but the two direct arms disagree with each other as much as with the original, and 6–10 questions per arm pair have stable, instance-specific law sets. The two-instance follow-up confirms that instance, not arm, drives these differences, and its output hashes locate the variation in the retrieval path itself: identical prepared inputs return identical law lists in only 65–79% of cross-instance cases. Quality comparisons of dispatch therefore need several instances per arm or a deterministic retrieval configuration. Expert relevance labels and a prespecified acceptable loss remain necessary, particularly for country comparison and case analysis.

A common question cohort gives exact balance for additive round and position effects, not carryover, interactions, or shared service traffic. The client runs separately, but per-request GPU memory effects and process contention are not quantified. Four active clients do not establish high-concurrency performance.

The rotated sweep settles what the exploratory pass could not. The mechanism scales as expected: the rewrite call costs 1.2–22 seconds per branch under load, and removing it cuts tool-engine queueing by 29–72%. The user-visible effect does not scale with it, because the orchestrator engine

saturates from concurrency 32 and absorbs the time released from the search branch; the pooled saving stays at 1.4–2.9% below saturation and 2.3–2.6% above it, with sign tests favoring dispatch at four of five levels and a mean interval that excludes zero only at 100. In this deployment, therefore, call removal is a small but consistent per-question improvement, and its value is bounded by the orchestrator’s capacity rather than by the removed work. Two rounds per level leave position effects of up to nine seconds at the saturation boundary, ten cells ran with a relaxed idle gate because of a stuck orchestrator request, production shared the engines, and output stability under saturation degraded for both arms; a longer sweep with more rounds and expert labels is needed before a quality claim at high load.

Planning should match the target. At the observed variance, 200 pairs per round have little power for a 0.586-second full-response effect. Adding repeated questions under the same queue can overstate effective sample size. Independent time blocks, randomized balanced order, workload controls, and a predefined primary endpoint are needed. The lower-variance search-completion endpoint should accompany, not replace, full-response latency.

Five timeouts are insufficient for reliability claims. All-attempt durations include incomplete responses; completed-pair analysis conditions on success. Both are reported. Fixed-concurrency throughput also depends strongly on response time and is not independent confirmation of speed.

Fewer short model invocations are the concrete service-load result: 41% fewer requests to the tool/answer engine per batch, but only 1–2% fewer prompt tokens, because the rewrite prompt is small compared with answer synthesis. The engine’s summed queue time also fell from about 9 seconds to near zero at four concurrent users. Whether this helps other concurrent requests at higher load remains untested. Future traces should record per-call native usage, complete output hashes, and linked queue, first-token, and final-token spans.

The adapter and reproducible analysis artifacts are planned for release at <https://github.com/choihyu/unmin/CNU-MultiAgent-RAG-Optimization>. Raw questions, responses, and service logs remain separate from aggregate analysis.

8. Conclusion

Across 1,800 requests, all 648 original search branches preserve prepared inputs. Direct dispatch removes those rewrite calls, lowers non-handler cost from 543.140 ms to 0.521 ms (0.880 ms with optional signing), and reduces requests to the tool/answer engine by 41% per batch while reducing prompt tokens by only 1–2%. A rotated sweep to concurrency 100 shows the removed rewrite call growing to 22 seconds per branch and tool-engine queueing falling by up to 72%, while the orchestrator engine saturates from concurrency 32 and bounds the end-to-end saving to 1.4–2.9% below saturation and 2.3–2.6% above it, with per-question sign tests favoring dispatch at every level except 32.

Time to the last search completion falls by 0.46–0.64 seconds across signed comparisons, against original-versus-original changes within 0.12 seconds. Full-response savings remain unresolved: the position-balanced estimate is 0.123 seconds with interval $[-1.15, 1.29]$, and original rounds differ by up to 1.5 seconds. Source-set agreement between arms is lower than each arm’s own repeatability, but two instances of the same arm agree no more than different arms, and identical inputs return identical law lists in only 65–79% of cross-instance cases; the gap is retrieval nondeterminism, not dispatch.

The contribution is a measured gap between local work, intermediate completion, and user-visible latency, with matched null controls at each scope. Call-count reduction is established; token savings are small; equal runtime outputs, unchanged legal accuracy, and deployment-wide acceleration are not established. Future studies need scope-matched endpoints, instance-balanced quality comparisons, complete output traces, and designs sized for the expected gain.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2005.11401>
- [2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023. <https://arxiv.org/abs/2210.03629>
- [3] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. <https://arxiv.org/abs/2302.04761>
- [4] Y. Qin et al., “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2307.16789>
- [5] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” in *Proc. COLM*, 2024. <https://arxiv.org/abs/2308.08155>
- [6] S. Kim et al., “An LLM Compiler for Parallel Function Calling,” in *Proc. ICML*, 2024. <https://arxiv.org/abs/2312.04511>
- [7] C. Lin et al., “Parrot: Efficient Serving of LLM-based Applications with Semantic Variable,” in *Proc. USENIX OSDI*, pp. 929–945, 2024. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [8] M. Luo et al., “Agentix: An Efficient Serving Engine for LLM Agents as General Programs,” in *Proc. USENIX NSDI*, pp. 2443–2459, 2026. <https://www.usenix.org/conference/nsdi26/presentation/luo>
- [9] L. Zheng et al., “SGLang: Efficient Execution of Structured Language Model Programs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. doi:10.52202/079017-2000.
- [10] C. Jin et al., “RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation,” arXiv:2404.12457, 2024. <https://arxiv.org/abs/2404.12457>
- [11] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, 1997. <https://www.rfc-editor.org/rfc/rfc2104>
- [12] A. Birgisson et al., “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in *Proc. NDSS*, 2014. doi:10.14722/ndss.2014.23212.
- [13] NIST/SEMATECH, “Latin square and related designs,” *e-Handbook of Statistical Methods*, Sec. 5.3.3.2.1. <https://www.itl.nist.gov/div898/handbook/pri/section3/pri3321.htm>
- [14] NIST/SEMATECH, “Sample sizes required,” *e-Handbook of Statistical Methods*, Sec. 7.2.2.2. <https://www.itl.nist.gov/div898/handbook/prc/section2/prc222.htm>